# COMP7610 Smart Contract Lab – Assignment 3

HKU Drone Club: CPT and Campus-Memo

Student: [REPLACE: Full name] UID: [REPLACE: UID] Network: Sepolia (Chain ID 11155111)

**System design.** `CPT` is an OpenZeppelin ERC-20 token with 18 decimals. Deployment mints exactly 1,000 CPT to the CPT contract as the club treasury. Only `ADMIN_ROLE` can call `mintPoints` (new points), `distributePoints` (treasury points), or `adminBurnPoints` (permanently deduct a specified amount from a user's balance). Standard `transfer`, `approve`, and `transferFrom` remain available to users.

Members call `applyForMembership`; an administrator calls `approveMembership`, which grants membership and transfers a fixed 5 CPT welcome allocation from the treasury. A member can call `withdrawFromClub` to burn all remaining CPT and revoke membership. Existing NFTs are not destroyed. This is the additional custom withdrawal function.

Administrators can also call `adminRecoverPoints(account, amount)` to return existing CPT to the contract treasury without user approval. Recovery preserves total supply, membership, and NFT ownership; recovered CPT can be redistributed.

`CampusMemo` is an ERC-721 collection named "Campus-Memo". Its maximum supply is 10 and minting costs 100 CPT. Only CPT administrators can issue NFTs: the administrator approves 100 CPT and calls `mintMemo(memberAddress)`. The recipient must be a member; 100 CPT is burned from the administrator, while the recipient's balance is unchanged. There is no NFT burn or administrator recovery function. Token IDs start at 1. The constructor stores one commit-pinned GitHub Raw image URL, and `tokenURI(id)` generates the JSON on-chain and returns a `data:application/json;base64,...` URI. No Gist or external JSON files are required. An owner can call `listMemo(tokenId, price)` to escrow an NFT, while a buyer approves the marketplace for the CPT price and calls `buyMemo`; CPT is paid to the seller and the NFT is transferred to the buyer. The seller can cancel with `cancelMemoListing`.

**Local verification.**

```
npm install
npm run compile
npm test
```

Remix: compile both contracts, select Remix VM, then run `scripts/remix-test.js`. The script uses fresh deployments and reports 22 test groups with balance, event, URI and revert checks. The local test suite also checks the initial treasury supply, administrator permissions, welcome points, CPT transfer, administrator CPT deduction, withdrawal burn, preservation of an already-owned NFT after withdrawal, administrator-only NFT issuance and payment burn, NFT transfer, CPT NFT purchase, listing cancellation, URI generation, and rejection of the 11th NFT.

**Sepolia deployment (Hardhat).**

1. In MetaMask select Sepolia and obtain test ETH. Never use a mainnet private key.
2. Set environment variables locally (do not commit them):

```
export SEPOLIA_RPC_URL="<RPC URL>"
export DEPLOYER_PRIVATE_KEY="<test-wallet private key>"
export NFT_IMAGE_URL="<full commit-pinned GitHub Raw image URL>"
```

3. Run `npm run compile`, then `npm test`, then `npm run deploy:sepolia`.
4. Record the printed addresses and deployment transaction hashes. Verify both addresses on Sepolia Etherscan.

**Remix + MetaMask function test.** Detailed steps are in `REMIX_DEPLOYMENT_GUIDE.md`.

1. Open the two Solidity files in Remix (OpenZeppelin 5.0.2); compile with Solidity 0.8.24, optimizer 200 and EVM paris, use a supported wallet connection (or Browser Extension in Remix web), and confirm Sepolia.
2. Deploy `CPT` with the deployer's address. Deploy `CampusMemo` with the CPT address and the verified GitHub Raw image URL.
3. As a student account call `applyForMembership`; as the administrator call `approveMembership(student)`. Confirm `isMember=true` and a 5 CPT balance.
4. As administrator call `adminRecoverPoints(student, 200000000000000000)` to recover 2 CPT. Confirm the student's balance decreases by 2, treasury balance increases by 2, and total supply and membership are unchanged. Non-admin calls must revert.
5. As administrator distribute 100 CPT to the admin account. In CPT call `approve` with the NFT address and amount `100000000000000000000`, then in CampusMemo call `mintMemo(student)`. Confirm ordinary-member minting reverts and the administrator pays 100 CPT. Confirm `ownerOf(1)` and `tokenURI(1)`.
6. From the student account call `transfer` to another account and `safeTransferFrom` for token 1. Confirm both new balances/owners.
7. As the NFT owner call `listMemo(1, price)`. From a buyer account approve the Campus-Memo contract to spend that CPT amount, then call `buyMemo(1)`. Confirm CPT reaches the seller and `ownerOf(1)` becomes the buyer. A seller can use `cancelMemoListing(1)` before purchase.
8. First return token 1 to the member. That member calls `withdrawFromClub()`. Confirm a zero CPT balance, `isMember=false`, a `MemberWithdrawn` event, and unchanged NFT ownership: `ownerOf(1)` is still the former member.

**Final submission values.**

| | |
|---|---|
| **CPT address** | [REPLACE: paste deployed Sepolia address] |
| **Campus-Memo address** | [REPLACE: paste deployed Sepolia address] |
| **NFT image URL** | [REPLACE: paste full commit-pinned GitHub Raw image URL] |
| **Decoded token JSON** | [REPLACE: paste the JSON decoded from tokenURI(1)] |
| **Example token URI** | [REPLACE: paste complete tokenURI(1) data URI] |
| **Deployment transactions** | [REPLACE: paste transaction hashes or Etherscan links] |

**Screenshots to insert before submission:** (1) MetaMask showing Sepolia and the deployed contracts; (2) membership approval and 5 CPT balance; (3) CPT approval followed by NFT mint, `ownerOf(1)`, and `tokenURI(1)`; (4) withdrawal transaction, zero CPT balance, `isMember=false`, and unchanged `ownerOf(1)`. Rename this file to `FullName_UID_Assignment3_Report.tex` and export the same content as PDF after replacing every red placeholder.